

Comprehensive 100 Interview Questions: Senior Java Developer

1. Advanced Java SE & Java EE (15 Questions)

1. **Java Memory Model & Concurrency:** Explain the `happens-before` guarantee provided by the Java Memory Model (JMM). How does it relate to `volatile`, `synchronized`, and final fields?
2. **Garbage Collection:** Compare the architectural differences and pause-time trade-offs between G1GC, ZGC, and Shenandoah. How would you diagnose and tune an application suffering from long GC pauses?
3. **Virtual Threads (Project Loom):** How do Java 21 Virtual Threads differ from OS-level platform threads in terms of scheduling, memory footprint, and thread pinning?
4. **Generics & Type Erasure:** Explain PECS (Producer Extends, Consumer Super). Why are generic arrays prohibited in Java, and how does type erasure affect reflection at runtime?
5. **CompletableFuture & Reactive Programming:** Compare asynchronous orchestration using `CompletableFuture` vs. Reactive Streams (e.g., Project Reactor). How do you handle exception propagation across threads?
6. **Classloaders & Modules:** Explain the classloading delegation hierarchy. How do custom classloaders cause memory leaks, and how did Java 9 Modules (JPMS) alter class accessibility?
7. **JVM Bytecode & Dynamic Proxies:** Contrast JDK Dynamic Proxies with ByteBuddy/CGLIB. How does bytecode manipulation work under the hood for runtime proxy creation?
8. **Java Collections Internals:** Explain the internal bucket structure, rehash mechanism, and red-black tree conversion threshold in `HashMap` (Java 8+). What causes concurrency issues when using `HashMap` instead of `ConcurrentHashMap`?
9. **Lock Free Data Structures:** Explain how `AtomicReference` uses Compare-And-Swap (CAS) instructions. What is the ABA problem and how does `AtomicStampedReference` address it?
10. **Custom Annotations & Reflection:** How do retention policies (`RUNTIME` vs `CLASS` vs `SOURCE`) work? What are the performance implications of extensive reflection usage in production code?
11. **Java EE Contexts & Dependency Injection (CDI):** How does CDI scope management (e.g., `@RequestScoped`, `@ApplicationScoped`) differ from Spring IoC bean scopes?
12. **JTA & Distributed Transactions:** Explain the Two-Phase Commit (2PC) protocol managed by a Java Transaction API (JTA) manager. What happens during a network partition phase?
13. **JMS & Message Driven Beans (MDB):** How does message acknowledgement work in JMS (e.g., `AUTO\_ACKNOWLEDGE` vs `CLIENT\_ACKNOWLEDGE`) within an EJB container?
14. **JPA/Hibernate Caching & Performance:** Explain the N+1 select problem. Compare `JOIN FETCH`, Entity Graphs, and Batch Fetching strategies for deep object graphs.
15. **Java Serialization & Vulnerabilities:** Why is native Java serialization considered dangerous? What mechanisms (like `readObject` validation or filter patterns) mitigate serialization attacks?

2. Comprehensive Spring Framework (20 Questions)

Spring IoC & Core

16. **Bean Lifecycle:** Trace the detailed lifecycle of a Spring Bean from instantiation, post-processors (`BeanPostProcessor`), custom init methods, to destruction.
17. **Circular Dependencies:** How does Spring resolve circular dependencies using the three-level cache architecture in `DefaultSingletonBeanRegistry`? When does it fail (e.g., constructor injection or `@Async`)?
18. **Custom Spring Starters:** How do you build a custom Spring Boot Auto-

Configuration module using `@ConditionalOnProperty`, `@ConditionalOnClass`, and `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`?

Spring MVC & Boot

19. ****Request Handling Flow:**** Trace a request from the servlet container through `DispatcherServlet`, `HandlerMapping`, `HandlerAdapter`, `HandlerInterceptor`, to `@ControllerAdvice`.
20. ****Spring Boot Startup Mechanics:**** What happens internally when `SpringApplication.run()` is invoked? How are `SpringFactoriesLoader` / auto-configurations executed?

Spring AOP

21. ****Aspect Architecture:**** Explain the difference between compile-time / load-time weaving (AspectJ) and dynamic proxy weaving (Spring AOP).
22. ****Self-Invocation Issue:**** Why doesn't `@Transactional` or `@Cacheable` trigger when a method calls another annotated method within the same bean, and how can this be resolved?

Spring Security

23. ****Security Filter Chain:**** Explain the order and execution of filters in `SecurityFilterChain`. How does `DelegatingFilterProxy` bridge Servlet container filters with Spring beans?
24. ****Authentication Mechanism:**** How does Spring Security authenticate requests using `AuthenticationManager`, `AuthenticationProvider`, and `SecurityContextHolder` (including thread propagation strategies)?
25. ****OAuth2 / OIDC:**** Describe how Spring Security implements the OAuth2 Authorization Code Grant with PKCE.

Spring Data & JDBC

26. ****Transaction Propagation:**** Explain the difference between `REQUIRED`, `REQUIRES_NEW`, `NESTED`, and `MANDATORY` propagation levels in `@Transactional`.
27. ****Transaction Isolation Levels:**** How do database isolation levels (`READ_COMMITTED`, `REPEATABLE_READ`, `SERIALIZABLE`) map to Spring transaction configuration, and how are phantom reads avoided?
28. ****Spring Data Custom Repositories:**** How does Spring Data JPA dynamically generate implementation classes at startup? How do you correctly extend repositories with custom native queries?
29. ****JdbcTemplate vs JPA:**** When is it architecturally preferable to use `JdbcTemplate` or MyBatis over Hibernate/Spring Data JPA in an enterprise application?

Spring Testing

30. ****Context Caching:**** How does `@SpringBootTest` cache the `ApplicationContext` across test classes, and what causes context dirties (`@DirtiesContext`)?
31. ****Testing Slice Annotations:**** Compare `@WebMvcTest`, `@DataJpaTest`, and `@JsonTest`. How do `@MockBean` and `@SpyBean` interact with the Spring context cache?
32. ****Testcontainers Integration:**** How do you set up dynamic database containerized environments using `@Testcontainers` and `@DynamicPropertySource` in Spring Boot 3+?
33. ****Mocking Static Methods & Private States:**** Why is mocking static methods generally discouraged, and how does modern Mockito / WireMock handle external HTTP dependency mocking?
34. ****Transactional Tests:**** Why do `@Transactional` annotations on test methods automatically roll back by default, and what false positives can this cause with JPA lazy loading?
35. ****Spring Security Testing:**** How do you test secured endpoints using `@WithMockUser`, `@WithSecurityContext`, and `SecurityMockMvcRequestPostProcessors`?

3. SQL & Oracle Database Management (15 Questions)

36. **Oracle Indexing Architecture:** Compare B-Tree, Bitmap, and Function-Based indexes. Under what conditions will Oracle's Cost-Based Optimizer (CBO) skip an index?
37. **Execution Plan Analysis:** How do you analyze an Oracle Execution Plan (`EXPLAIN PLAN`, `V$SQL_PLAN`, `AUTOTRACE`)? What do `TABLE ACCESS FULL`, `INDEX RANGE SCAN`, and `NESTED LOOPS` signify?
38. **Oracle Partitioning:** Compare Range, List, Hash, and Composite partitioning. How does partition pruning improve query performance on multi-terabyte tables?
39. **Analytic (Window) Functions:** Explain the difference between `RANK()`, `DENSE_RANK()`, and `ROW_NUMBER()`. Write a query logic using `OVER(PARTITION BY ... ORDER BY ... ROWS BETWEEN ...)`.
40. **PL/SQL Subprograms & Packages:** How do PL/SQL Cursor Variables (`REF CURSOR`), Bulk Collect, and `FORALL` statements minimize context switches between the PL/SQL engine and SQL engine?
41. **Locking & Concurrency:** Explain Oracle's multi-version concurrency control (MVCC). How does Oracle prevent dirty reads without using shared locks? What causes `ORA-00060: deadlock detected`?
42. **Flashback & Undo Tablespace:** How does Oracle use the `UNDO` tablespace to support Flashback Query (`AS OF TIMESTAMP`) and read consistency?
43. **Hierarchical Queries:** Explain how to write recursive queries using Oracle's `CONNECT BY PRIOR` vs. ANSI SQL Common Table Expressions (`WITH clause`).
44. **High Availability (RAC & Data Guard):** Explain the architecture of Oracle Real Application Clusters (RAC) vs Oracle Data Guard (Primary/Standby). How does application failover work via Fast Connection Failover (FCF)?
45. **Database Maintenance & Statistics:** Why is gather table statistics (`DBMS_STATS.GATHER_TABLE_STATS`) vital for the CBO? How does stale statistics degrade execution plans?
46. **MERGE Statements:** Write and explain an upsert operation using the SQL `MERGE INTO` syntax, ensuring handling of matching and non-matching records cleanly.
47. **Temporary Tables & LOB Handling:** Compare Global Temporary Tables (GTT) with normal tables. How should `CLOB` / `BLOB` fields be handled efficiently in Java/Oracle applications to avoid memory exhaustion?
48. **Query Optimization:** How do hints like `/*+ INDEX(t idx_name) */` or `/*+ LEADING(a) USE_NL(b) */` influence the CBO? What are the risks of using explicit optimizer hints?
49. **Pessimistic vs Optimistic Locking:** How do you implement pessimistic locking (`SELECT ... FOR UPDATE NOWAIT`) in Oracle via JPA/Spring Data, and how do you handle `ORA-00054` exceptions?
50. **Materialized Views:** What is the difference between `ON COMMIT` and `ON DEMAND` refresh for Materialized Views? What is fast refresh via Materialized View Logs?

4. Servlets, Tomcat, and WebLogic Environments (15 Questions)

51. **Servlet Lifecycle:** Explain `init()`, `service()`, and `destroy()`. How does a servlet container maintain thread safety when multiple concurrent HTTP requests hit a single Servlet instance?
52. **Filter vs Servlet vs Listener:** Compare `javax/jakarta.servlet.Filter`, `HttpServlet`, and `ServletContextListener`. What is the execution hierarchy during startup and request processing?
53. **Async Servlet Processing:** How does asynchronous servlet processing (`AsyncContext`, `startAsync()`) free up container threads during long-running I/O operations?
54. **Session Management & Clustering:** How are HTTP Sessions stored, updated, and replicated across Tomcat/WebLogic cluster nodes? What is the impact of session stickiness vs session replication?
55. **Tomcat Architecture:** Explain the hierarchy of Tomcat components:

`Server` -> `Service` -> `Engine` -> `Host` -> `Context` -> `Connector`.

56. ****Tomcat Thread Pool Tuning:**** How do parameters like `maxThreads`, `minSpareThreads`, `acceptCount`, and `maxConnections` impact performance under high traffic?

57. ****Connectors (NIO vs APR):**** What is the difference between Tomcat's HTTP/NIO, HTTP/NIO2, and APR (Apache Portable Runtime) connectors?

58. ****WebLogic Domain Architecture:**** Explain the roles of the Administration Server, Managed Servers, Node Manager, and WebLogic Cluster.

59. ****WebLogic Work Managers:**** How do WebLogic Work Managers customize thread scheduling (`Fair Share Request Class`, `Response Time Request Class`, `Constrained Request Class`) compared to standard thread pools?

60. ****JNDI & DataSource Management:**** How are DataSources and Connection Pools (e.g., HikariCP on Tomcat vs WebLogic GridLink Data Sources) configured, monitored, and tuned?

61. ****Classloading in WebLogic:**** Explain WebLogic's hierarchical classloading mechanism. How does `weblogic-application.xml` (using `prefer-application-packages`) solve library dependency conflicts?

62. ****Shared Libraries & EAR Deployment:**** Compare WAR vs. EAR deployments. How do shared libraries (`manifest.mf` Class-Path or WebLogic Shared Libraries) work across multiple EAR deployments?

63. ****Garbage Collection Logs in Containers:**** How do you enable and extract verbose GC logs and thread dumps from a remote Tomcat/WebLogic container in a production failure scenario?

64. ****WebLogic Security Realms:**** How do WebLogic Security Realms integrate with LDAP / Active Directory for enterprise Authentication and Authorization?

65. ****Graceful Shutdown:**** What happens behind the scenes during a graceful shutdown of Tomcat or WebLogic? How do active HTTP requests complete before the container terminates?

5. Linux, Bash, and CI/CD (TeamCity & Jenkins) (20 Questions)

Linux System Administration & Bash

66. ****File Permissions & ACLs:**** Explain standard Linux permissions (`chmod 755`, `chown`, `chgrp`), `umask`, Special Permissions (SUID, SGID, Sticky Bit), and POSIX ACLs (`getfacl`, `setfacl`).

67. ****Process & Resource Monitoring:**** How do you troubleshoot high CPU or memory consumption using `top`, `htop`, `ps aux`, `pidstat`, `vmstat`, and `free`?

68. ****Network Diagnostics:**** How do you check listening ports, open sockets, and network routes using `netstat`, `ss`, `lsof`, `tcpdump`, `curl`, and `nc` (netcat)?

69. ****System Logs & Journald:**** How do you inspect and filter system logs using `journalctl`, `/var/log/syslog`, and text-processing pipelines (`grep`, `awk`, `sed`, `less`)?

70. ****Process & Service Handling:**** Compare Systemd vs SysVinit. How do you create, start, enable, and inspect a custom Systemd unit file (`my-java-app.service`)?

71. ****Security Keys & TLS Certificates:**** How do you inspect, generate, and convert SSL/TLS certificates using `openssl` (e.g., `.crt`, `.pem`, `.jks`, `.p12`)? How do Java KeyStore (`keytool`) and Linux certificates interact?

72. ****Bash Scripting - Error Handling:**** Write a robust Bash script pattern that uses `set -e`, `set -u`, `set -o pipefail`, and custom `trap` handlers for cleanup on failure.

73. ****Bash Text Processing:**** Write a command pipeline using `awk` and `sed` to extract specific metrics from an application log file and sort the top 5 occurrences.

74. ****User & Group Administration:**** How do you safely create users, add them to specific groups, handle SSH key distribution (`~/.ssh/authorized\_keys`), and configure `/etc/sudoers`?

75. ****Storage & Disk Space:**** How do you identify disk space hogs and inode exhaustion using `df -h`, `du -sh`, and `find`?

CI/CD Pipelines (TeamCity & Jenkins)

76. ****Jenkins Pipeline Architecture:**** Compare Declarative vs Scripted Jenkins pipelines. How do `agent`, `stages`, `steps`, `post`, and `environment` blocks function in a `Jenkinsfile`?
77. ****Jenkins Shared Libraries:**** How do you create and use reusable Jenkins Shared Libraries (Groovy) across enterprise projects?
78. ****TeamCity Build Configurations:**** Explain the concepts of Build Steps, Build Triggers, Artifact Dependencies, and Snapshot Dependencies in TeamCity.
79. ****TeamCity Build Chains:**** How do build chains allow parallel execution, pipeline visualization, and reuse of build artifacts across heterogeneous build agents?
80. ****Build Agent Management:**** How do TeamCity Agent Pools and Jenkins Controller-Agent architectures scale build throughput? How are build environment capabilities matched to job requirements?
81. ****Pipeline Security & Secrets Management:**** How do you securely inject credentials (passwords, SSH keys, tokens) into Jenkins/TeamCity pipelines without leaking them to build logs?
82. ****Artifact Management:**** How do CI tools integrate with enterprise artifact repositories (Nexus, JFrog Artifactory) for versioning compiled JAR/WAR artifacts and Docker images?
83. ****Automated Testing & Quality Gates:**** How do you integrate static analysis tools (SonarQube) and code coverage tools (JaCoCo) into CI/CD pipelines to enforce quality gates before deployment?
84. ****Zero Downtime Deployment Strategies:**** Compare Blue/Green, Canary, and Rolling deployment strategies. How do CI pipelines automate these deployments to application servers?
85. ****Pipeline Caching:**** How do you configure caching of Maven (`~/.m2`) or Gradle dependency caches in TeamCity and Jenkins to accelerate build times?

6. Soft Skills: Analytical Problem-Solving & Team Consensus (10 Questions)

86. ****Complex Incident Root Cause Analysis:**** Describe a scenario where a production system crashed due to an intermittent issue (e.g., memory leak, thread starvation). Walk through your analytical step-by-step diagnostic process.
87. ****Architectural Trade-offs:**** Tell me about a time you had to choose between a quick temporary fix and a complex, maintainable long-term architectural solution under heavy business pressure. How did you decide?
88. ****Handling Technical Debt:**** How do you identify, prioritize, and articulate the business value of refactoring deep legacy code or upgrading framework dependencies to non-technical stakeholders?
89. ****Building Team Consensus:**** Describe a situation where senior engineers on your team had strongly conflicting opinions regarding an architectural decision (e.g., Microservices vs Monolith, ORM vs Raw SQL). How did you facilitate alignment?
90. ****Mentorship & Code Reviews:**** How do you handle code reviews where a peer's solution works functionality-wise but violates clean code or architectural standards? How do you give constructive feedback without causing friction?
91. ****Debugging Unknown Systems:**** You are assigned to fix a critical performance bug in a undocumented legacy codebase using technology you aren't deeply familiar with. What is your approach?
92. ****Fostering Ownership:**** How do you encourage a team culture where developers take collective ownership of build failures, quality control, and operational monitoring?
93. ****Post-Mortem Facilitation:**** Describe your process for running a Blameless Post-Mortem after a major outage. How do you turn human error into systemic improvements?
94. ****Disagree and Commit:**** Can you share an instance where you strongly disagreed with a technical direction chosen by your team or leadership, but ultimately had to support it? How did you manage it?
95. ****Communicating Risk:**** How do you present potential technical risks (such as database migration risks or third-party API deprecation) to project managers?

to get them prioritized on the product roadmap?

7. Senior-Level System Design & Architecture Scenarios (5 Questions)

96. **Legacy Migration:** How would you plan and execute the gradual migration of a monolithic Java EE application running on WebLogic/Oracle to a cloud-native Spring Boot microservices architecture using the Strangler Fig pattern?
 97. **High-Throughput Ingestion:** Design a system capable of handling 50,000 requests/sec with low latency, writing data to Oracle while keeping search queries sub-millisecond using caching layers.
 98. **Distributed Transaction Management:** How do you maintain data consistency across multiple Spring Boot microservices without using traditional 2PC/JTA (e.g., using the Saga Pattern, Outbox Pattern, and CDC)?
 99. **Zero-Downtime Database Schema Migration:** How do you execute schema alterations (e.g., renaming a heavily queried column in an Oracle production table) without downtime using Liquibase/Flyway and expand/contract patterns?
 100. **Resilience & Fault Tolerance:** How do you design a Spring Boot enterprise app resilient to cascading downstream failures using Circuit Breakers (Resilience4j), Rate Limiters, Bulkheads, and Retry strategies?
- ```eof

Summary of Material Provided

I have compiled a comprehensive list of **100** high-level technical and soft-skills questions tailored specifically for a **Senior Java Developer** role with 10+ years of experience. The material is organized into seven structural pillars matching your exact requirements:

1. **Java SE & Java EE:** Advanced concurrency, memory management, JVM internals, virtual threads, and enterprise spec architecture.
2. **Spring Framework:** Deep dive into IoC, Spring Boot internals, AOP dynamic proxies, Spring Security execution chains, Spring Data transactions, and slice testing.
3. **SQL & Oracle DB:** Optimizer execution plans, B-Tree/Bitmap indexes, analytic window functions, PL/SQL bulk operations, MVCC, and performance tuning.
4. **Servlets, Tomcat & WebLogic:** Lifecycle execution, container thread pool parameters, WebLogic Work Managers, cluster replication, and JNDI pool management.
5. **Linux, Bash & CI/CD:** Diagnostics, POSIX permissions, log parsing, shell scripts, Jenkins pipelines (Groovy), TeamCity build chains, and release strategies.
6. **Soft Skills & Consensus:** Scenario questions focusing on problem-solving under pressure, leading technical debates, blameless post-mortems, and technical debt management.
7. **System Design & Architecture:** Enterprise-scale architectural problems covering migration patterns, high-throughput ingestion, distributed sagas, and zero-downtime DB migrations.

Good luck with your preparation for the interview!